

Laporan Proyek Akhir Praktikum OOP



Kelompok 17:

Naufal Rahman (2406413142)

Raihan Muhammad Nafis Al-Kautsar (2406413451)

Dimar Ilham Tamara (2406413804)

Ahmad Malik Prasetyo (2406416270)

Game Identity / Mantra:

Floor is Lava adalah game 2D di mana player harus melompat dari satu platform ke platform lainnya yang posisinya muncul secara acak, namun tetap dirancang agar selalu dapat dijangkau oleh player. Sepanjang permainan, akan ada lava yang muncul dari bawah dan terus naik ke atas dan kecepatannya akan meningkat menyesuaikan kemampuan pemain bermain. Semakin jago pemainnya, semakin meningkat kecepatan lavanya. Pemain juga dapat melakukan dashing ketika lavanya terlalu dekat dengan pemainnya untuk ke platform di atasnya. Selain itu, koin juga tersebar di sepanjang jalur untuk dikumpulkan sebagai poin tambahan. Kamera game akan tetap berada pada posisi horizontal yang sama, tetapi akan mengikuti pergerakan vertikal player ke atas.

Design Pillars:

Proyek ini dibangun di atas fondasi **Object-Oriented Programming (OOP)** yang kuat dan menerapkan berbagai **Software Design Patterns** standar industri. Struktur kode diorganisir ke dalam paket-paket (*packages*) yang terpisah berdasarkan tanggung jawabnya (*Separation of Concerns*).

1. Penerapan 4 Pilar Utama Pemrograman Berbasis Objek (OOP Principles)

Pilar ini adalah pondasi yang diterapkan di seluruh kode program untuk menjamin modularitas dan kemudahan pemeliharaan (*maintainability*).

a. Abstraction (Abstraksi)

- **Konsep:** Menyembunyikan kompleksitas implementasi detail dan hanya mengekspos fungsionalitas utama melalui "kontrak" berupa Interface atau Abstract Class.
- **Implementasi di Project:**
 - Terlihat jelas pada penggunaan **Interface** (ikon **I** hijau di IDE). Contoh utamanya adalah file **GameState.java** di package **states**. Kelas lain tidak perlu tahu detail bagaimana sebuah state di-render, mereka hanya perlu tahu bahwa setiap state memiliki method **update()** dan **render()**.

- Pada package `commands`, terdapat file `Command.java` yang merupakan abstraksi untuk semua aksi player.
- Pada package `observers`, terdapat file `Observer.java` dan `Subject.java` yang menjadi kerangka dasar sistem notifikasi tanpa terikat pada implementasi konkret UI tertentu.
- **Tujuannya:** Memungkinkan sistem untuk berkembang tanpa merusak kode yang sudah ada, karena komponen bergantung pada abstraksi, bukan implementasi konkret.

b. Encapsulation (Enkapsulasi)

- **Konsep:** Membungkus data (variabel) dan metode menjadi satu unit kelas, serta melindungi akses langsung ke data sensitif menggunakan *access modifiers* (`private`, `protected`).
- **Implementasi di Project:**
 - Pada file `Player.java`, atribut seperti posisi koordinat (`Vector2`), status hidup/mati, dan kecepatan gerak pasti diatur sebagai `private`. Akses dari luar (misalnya dari `PlayingState.java`) dilakukan melalui *public methods* atau *getter/setter* yang terkontrol.
 - Pada file `GameManager.java`, data global seperti skor dan koin dilindungi agar tidak bisa diubah sembarangan oleh kelas lain selain melalui method resmi seperti `addCoin()` atau `setScore()`.
- **Tujuannya:** Menjaga integritas data agar status permainan tidak menjadi *corrupt* akibat perubahan variabel yang tidak valid dari luar kelas.

c. Inheritance (Pewarisan)

- **Konsep:** Mekanisme di mana sebuah kelas baru mewarisi sifat dan perilaku dari kelas induk (*parent class*) untuk mengurangi duplikasi kode (*code reuse*).
- **Implementasi di Project:**
 - Dapat dilihat pada package `strategies`. File `VeryEasyDifficulty.java`, `MediumDifficulty.java`, hingga `ExtremeDifficulty.java` mewarisi logika dasar dari kelas induk

(**DifficultyStrategy**). Mereka berbagi properti yang sama tapi memiliki nilai parameter (kecepatan lava) yang berbeda.

- Pada package **pools**, file **CoinPool.java** dan **GroundsPool.java** mewarisi logika manajemen memori dari file induk **ObjectPool.java**.
- **Tujuannya:** Efisiensi penulisan kode. Logika dasar (seperti cara kerja *pooling*) cukup ditulis satu kali di kelas induk, dan kelas anak hanya perlu menyesuaikan tipe objeknya saja.

d. Polymorphism (Polimorfisme)

- **Konsep:** Kemampuan objek untuk diperlakukan sebagai bentuk umumnya, namun perilakunya tetap mengikuti bentuk spesifiknya saat dijalankan (*runtime*).
- **Implementasi di Project:**
 - Pada **GameStateManager.java**, sistem memanggil method **render()** pada variabel bertipe **GameState**. Saat *runtime*, jika objek aslinya adalah **MenuState.java**, maka tampilan menu yang muncul. Jika objeknya **PlayingState.java**, maka gameplay yang muncul.
 - Pada **PlayingState.java**, terdapat list **List<Command>**. Saat perulangan (*loop*), program memanggil **command.execute()**. Perintah **LompatCommand.java** akan membuat player melompat, sedangkan **PauseCommand.java** akan menghentikan game, meskipun keduanya dipanggil dengan cara yang sama.
- **Tujuannya:** Fleksibilitas kode. Kita bisa menambahkan tipe State baru atau Command baru tanpa mengubah logika utama pengelola game.

2. Implementasi Software Design Patterns

Berdasarkan struktur folder, proyek ini menerapkan pola desain yang sangat rapi:

- **State Pattern (Package **com.dimar.frontend.states**)**
 - **File Terkait:** **GameState.java**, **GameStateManager.java**, **LoginState.java**, **MenuState.java**, **PlayingState.java**, **PauseState.java**, **GameOverState.java**, **RegisterState.java**.
 - **Penjelasan:** Pola ini digunakan untuk mengelola alur permainan. Setiap fase permainan (Login, Menu, Main, Pause) dipisahkan menjadi kelas sendiri.

`GameStateManager` bertugas menumpuk (*push*) atau mengganti (*set*) state aktif. Ini mencegah kode bercampur, di mana logika menu bercampur dengan logika permainan.

- **Command Pattern (Package `com.dimar.frontend.commands`)**

- **File Terkait:** `Command.java`, `DashKananCommand.java`, `DashKiriCommand.java`, `KananCommand.java`, `KiriCommand.java`, `LompatCommand.java`, `PauseCommand.java`, `ResumeCommand.java`.
- **Penjelasan:** Pola ini mengenkapsulasi permintaan input menjadi objek. Input keyboard tidak langsung mengubah posisi player secara *hardcode*, melainkan memicu objek `Command`. Ini memudahkan fitur seperti *remapping* tombol atau menunda eksekusi perintah.

- **Factory Method & Object Pool Pattern (Package `factories & pools`)**

- **File Terkait:** `CoinFactory.java`, `GroundsFactory.java`, `CoinPool.java`, `GroundsPool.java`, `ObjectPool.java`.
- **Penjelasan:**
 - **Factory:** Digunakan untuk menyembunyikan logika pembuatan objek `Coin` dan `Grounds` yang kompleks.
 - **Object Pool:** Game ini sering memunculkan dan menghapus objek. Daripada terus-menerus membuat (*new*) dan menghancurkan objek (yang membebani *Garbage Collector* dan bikin *lag*), pola ini mendaur ulang objek yang sudah tidak terpakai. `CoinPool` menyimpan koin yang hilang untuk dipakai lagi nanti.

- **Strategy Pattern (Package `strategies & Root Classes`)**

- **File Terkait:** `VeryEasyDifficulty.java`, `EasyDifficulty.java`, `MediumDifficulty.java`, `HardDifficulty.java`, `VeryHardDifficulty.java`, `ExtremeDifficulty.java`, `LinePattern.java`.
- **Penjelasan:** Pola ini memungkinkan algoritma atau perilaku game diubah saat *runtime*. Saat pemain sangat jauh dari lava, strategi kesulitan diganti dari `Easy` ke `Medium` (yang mengubah kecepatan lava & musik), tanpa perlu mengubah kode inti di `PlayingState`. Semakin jauh pemain dari lava, semakin meningkat tingkat kesulitannya karena menandakan pemainnya bermain dengan jago.

Selain itu, `LinePattern.java` juga menunjukkan strategi pola kemunculan koin.

- **Observer Pattern (Package `observers`)**

- **File Terkait:** `Observer.java`, `Subject.java`, `ScoreManager.java`, `ScoreUIObserver.java`, `DashUI.java`.
- **Penjelasan:** Pola ini membuat hubungan *subscription*. `ScoreUIObserver` (UI Skor) "berlangganan" ke `GameManager` atau `ScoreManager` (Subject). Ketika skor berubah, Subject memberi notifikasi ke Observer untuk update tampilan. Ini memisahkan logika *backend* data dari logika *frontend* tampilan.

- **Singleton Pattern**

- **File Terkait:** `GameManager.java` (Root package).
- **Penjelasan:** `GameManager` dirancang agar hanya memiliki satu *instance* di seluruh aplikasi. Ini bertindak sebagai pusat akses global untuk data sesi pemain, skor, dan koin yang harus konsisten di semua State. Selain itu, `GameManager` juga memiliki metode-metode yang berhubungan dengan `BackendService` untuk melakukan fetching dengan backend.

3. Organisasi Aset & Layanan (Assets & Services)

- **Service Layer (Package `services`)**

- **File Terkait:** `BackendService.java`.
- **Penjelasan:** Memisahkan logika bisnis (seperti request API untuk Login/Register) dari logika game. Ini membuat kode UI (`LoginState`) bersih dari kode jaringan HTTP.

- **Structured Asset Management (Folder `assets`)**

- **Folder Terkait:** `audio` (sound/music), `dash`, `login`, `menu`, `pause`, `player`, `register`.
- **File Terkait:** `04b30.fnt`, `background.jpg`, `Cobblestone.jpg`, dan lainnya.
- **Penjelasan:** Aset dikelompokkan berdasarkan fungsinya, memudahkan pengelolaan sumber daya. Penggunaan font bitmap (`.fnt`) dan tekstur terpisah menunjukkan manajemen aset grafis yang rapi untuk framework LibGDX.

Design Patterns:

Pengembangan game ini menerapkan berbagai pola desain perangkat lunak (*Software Design Patterns*) untuk memastikan kode yang modular, efisien, dan mudah dipelihara.

1. Singleton Pattern

- **Konsep:** Pola ini memastikan bahwa sebuah kelas hanya memiliki satu *instance* (objek) yang dibuat sepanjang siklus hidup aplikasi dan menyediakan titik akses global ke objek tersebut.
- **Implementasi di Project:** Diterapkan pada kelas **GameManager**. Game ini membutuhkan satu pengelola pusat yang menyimpan data sesi pemain (token otentikasi), skor, dan koin yang harus konsisten dan dapat diakses dari berbagai *State* (**MenuState**, **PlayingState**, dll) tanpa perlu membuat objek baru berulang kali.
- **Bukti Kode (GameManager.java):**

```
public class GameManager {  
    private static GameManager instance; // Menyimpan  
    satu-satunya instance  
  
    // Constructor private agar tidak bisa di-instansiasi dari  
    luar  
    private GameManager() {  
        scoreManager = new ScoreManager();  
        backendService = new BackendService();  
        // ... inisialisasi lainnya  
    }  
  
    // Method akses global  
    public static GameManager getInstance() {  
        if (instance == null) {  
            instance = new GameManager();  
        }  
        return instance;  
    }  
}
```

2. Factory Method Pattern

- **Konsep:** Pola yang menyediakan interface untuk membuat objek, tetapi membiarkan subclass atau kelas pabrik (*factory*) memutuskan detail instansiasinya. Ini menyembunyikan kompleksitas pembuatan objek dari kode utama.
- **Implementasi di Project:** Diterapkan pada **CoinFactory**. Kelas **PlayingState** tidak perlu mengetahui detail teknis bagaimana objek koin dibuat atau dikelola di memori. Ia hanya perlu meminta **CoinFactory** untuk menyediakan list koin yang aktif.
- **Bukti Kode (CoinFactory.java):**

```
public class CoinFactory {  
    public final CoinPool coinPool = new CoinPool();  
  
    // Menyembunyikan logika pengelolaan memori (pool) dari  
    gameplay utama  
    public void release(Coin coin) {  
        coinPool.release(coin);  
    }  
  
    public List<Coin> getInUse() {  
        return coinPool.getInUse();  
    }  
}
```

3. Object Pool Pattern

- **Konsep:** Pola optimasi memori yang mendaur ulang objek yang sudah tidak terpakai alih-alih menghancurkan dan membuatnya kembali (*create & destroy*). Dalam pengembangan game, ini penting untuk mencegah *Garbage Collector* yang dapat menyebabkan *frame drop* (lag).
- **Implementasi di Project:** Diimplementasikan melalui kelas abstrak **ObjectPool<T>** yang kemudian diturunkan ke **CoinPool**. Ketika koin diambil oleh pemain atau keluar dari layar, objek tersebut tidak dihapus (*null*), melainkan dikembalikan ke list **available** untuk digunakan kembali nanti.

- **Bukti Kode (ObjectPool.java):**

```
public abstract class ObjectPool<T> {
    private final List<T> available = new ArrayList<>();
    private final List<T> inUse = new ArrayList<>();

    public T obtain() {
        T object;
        if (available.isEmpty()) {
            object = createObject(); // Buat baru jika stok
kosong
        } else {
            object = available.remove(available.size() - 1); //
Pakai ulang (recycle)
        }
        inUse.add(object);
        return object;
    }

    public void release(T object) {
        inUse.remove(object);
        resetObject(object); // Reset status objek
        available.add(object); // Simpan untuk nanti
    }
}
```

4. Command Pattern

- **Konsep:** Mengenkapsulasi sebuah permintaan (request) atau aksi menjadi sebuah objek mandiri. Ini memungkinkan parameterisasi objek dengan operasi yang berbeda.
- **Implementasi di Project:** Digunakan untuk menangani Input Pemain. Aksi "Lompat" dibungkus dalam kelas **LompatCommand** yang mengimplementasikan interface **Command**. Hal ini membuat logika input terpisah dari logika karakter, dan memudahkan jika nanti ingin mengubah tombol kontrol (*remapping*).

- **Bukti Kode (Command.java & LompatCommand.java):**

```
// Interface Command

public interface Command {

    void execute();

}

// Implementasi Konkret

public class LompatCommand implements Command {

    private final Player player;

    public LompatCommand(Player player) {

        this.player = player;

    }

    @Override

    public void execute() {

        if (!player.getIsDead() &&

Gdx.input.isKeyJustPressed(Input.Keys.SPACE)) {

            player.Lompat();

        }

    }

}
```

5. State Pattern

- **Konsep:** Memungkinkan objek mengubah perilakunya ketika status internalnya berubah. Pada game ini, pola ini digunakan untuk manajemen layar.
- **Implementasi di Project:** Dikelola oleh **GameStateManager** yang mengatur transisi antar halaman game yang berbeda (seperti Menu, Gameplay, Pause). Setiap halaman mengimplementasikan interface **GameState**, sehingga Manager dapat memperlakukan semua halaman secara seragam (Polimorfisme) saat memanggil method **update()** atau **render()**.

- **Bukti Kode (GameStateManager.java & GameState.java):**

```
// Interface State

public interface GameState {
    void update(float delta);
    void render(ShapeRenderer shapeRenderer, SpriteBatch
spriteBatch);
}

// Context Class (Manager)
public class GameStateManager {
    private final Stack<GameState> states;

    public void set(GameState state) {
        if (!states.isEmpty()) {
            states.pop().dispose(); // Hapus state lama
        }
        states.push(state); // Masukkan state baru
    }

    public void render(ShapeRenderer shapeRenderer, SpriteBatch
batch) {
        states.peek().render(shapeRenderer, batch); // Render
state terbatas
    }
}
```

6. Observer Pattern

- **Konsep:** Mendefinisikan ketergantungan *one-to-many* antar objek, di mana jika satu objek (Subject) berubah statusnya, semua objek yang bergantung padanya (Observer) akan diberitahu dan diperbarui secara otomatis.
- **Implementasi di Project:** Digunakan pada sistem UI Skor. **ScoreUIObserver** bertindak sebagai *Observer* yang memantau perubahan data skor. Ketika skor berubah

di `GameManager` (melalui `ScoreManager`), UI otomatis mendapatkan notifikasi melalui method `update()` untuk memperbarui tampilan teks di layar.

- **Bukti Kode (`Observer.java` & `ScoreUIObserver.java`):**

```
// Interface Observer

public interface Observer {
    void update(int score);
}

// Implementasi Observer pada UI
public class ScoreUIObserver implements Observer {
    @Override
    public void update(int score) {
        this.score = score; // Update data lokal saat notifikasi
        diterima
    }

    public void render(int score, int coins, String lavaMode) {
        // Logika menggambar UI skor ke layar
        font.draw(batch, "Score: " + score, ...);
    }
}
```

7. Strategy Pattern

- **Konsep:** Pola yang digunakan untuk mendefinisikan sekumpulan algoritma yang dapat saling dipertukarkan. Setiap strategi dikemas dalam kelas terpisah dan dapat dipilih atau diganti secara dinamis saat runtime tanpa mengubah kode utama yang menggunakannya.
- **Implementasi di Project:** Strategi Pattern digunakan pada sistem pengaturan difficulty permainan. Strategi difficulty dapat berubah secara dinamis berdasarkan jarak pemain terhadap lava sehingga tingkat kesulitan dapat menyesuaikan kemampuan pemain selama permainan berlangsung.

- **Bukti Kode (DifficultyStrategy.java & MediumDifficulty.java):**

```
public interface DifficultyStrategy{
    float getKecepatanLava();
    float getBatasNaikDifficulty();
    DifficultyStrategy getNextDifficulty();
    String getMode();
    float getTimerToNext();
}

package com.dimar.frontend.strategies;
public class MediumDifficulty implements DifficultyStrategy {
    @Override
    public float getKecepatanLava() {
        return 210f;
    }

    @Override
    public float getBatasNaikDifficulty() {
        return 800f;
    }

    @Override
    public DifficultyStrategy getNextDifficulty() {
        return new HardDifficulty();
    }

    @Override
    public String getMode() {
        return "Medium";
    }

    @Override
```

```

    public float getTimerToNext() {
        return 20f;
    }
}

```

Genre/Story/Mechanics Summary:

Istana Amara Land kini dalam bahaya besar! Gunung Likam yang tertidur lama di bawahnya telah bangkit, memuntahkan lautan magma yang mulai menelan lantai dasar istana tempat Pangeran Ramid tertidur. Bantulah Ramid untuk segera bangun dan berlari, karena nasib ia sepenuhnya berada di tanganmu. Pandu dia memanjat reruntuhan pilar yang terbakar dalam pelarian vertikal yang memacu adrenalin, sebelum ia tenggelam dalam panasnya lautan api yang terus mengejar dari bawah.

Namun, jangan khawatir, kamu dibekali senjata rahasia: Sepatu Boots Nahiar. Manfaatkan teknologi ini untuk mengubah ancaman menjadi kekuatan! Rasakan ketegangannya saat kamu harus membiarkan lava mendekat hingga jarak kritis demi mengaktifkan daya ledak sepatu tersebut. Gunakan momentum itu untuk melakukan *dash* menembus jurang yang mustahil, karena hanya keberanianmu mengambil risiko yang bisa menyelamatkan Ramid dari maut.

Tujuanmu jelas: mencapai The Lafuan, puncak tertinggi istana tempat helikopter penyelamat satu-satunya menunggu di tengah kepulan asap. Abaikan rasa takutmu, fokuskan pandangan ke atas, dan jangan pernah menoleh ke bawah. Waktu terus berjalan dan lava semakin tinggi. Ayo, bantulah Ramid meloloskan diri dari neraka ini sekarang juga!

Mekanisme Game:

- Tekan A untuk ke kiri
- Tekan D untuk ke kanan
- Tekan Spasi untuk lompat
- Tekan Q untuk dash ke kiri
- Tekan E untuk dash ke kanan

Note: Dash hanya bisa dieksekusi ketika lava sudah terlalu dekat Ramid.

Features:

1. Infinite Runner Gameplay

Game Floor Is Lava menerapkan konsep infinite runner di mana permainan tidak memiliki batas akhir level. Platform akan terus dihasilkan secara acak seiring pergerakan pemain ke arah vertikal.

Platform yang dihasilkan tetap mempertimbangkan jarak lompatan maksimum pemain dan kecepatannya sehingga setiap platform yang muncul secara acak tetap dapat dicapai secara logis. Dengan desain ini, permainan dapat berlangsung tanpa henti selama pemain mampu menjaga jarak aman dari lava yang terus meningkat.

Dengan perhitungan Gerak Parabola, diturunkan rumus jarak horizontal antara platform di bawah dan di atasnya sehingga pemain selalu dapat mencapai platform tersebut.

$$d = v_x \left(\frac{v_y}{g} + \sqrt{\frac{2(l_p - (g_{platform} + h_{platform}))}{g}} \right)$$

di mana:

d = jarak horizontal platform bawah dan platform acuan di atasnya

$$l_p = \frac{v_y^2}{2g}$$

v_y = kecepatan lompatan player

v_x = kecepatan horizontal player

$g_{platform}$ = jarak vertikal antar platform

$h_{platform}$ = tinggi platform

g = percepatan gravitasi

2. Mekanisme Dashing

Fitur dashing memungkinkan pemain melakukan gerakan cepat ke arah platform acuan di atasnya dalam waktu singkat. Mekanisme ini hanya dapat digunakan ketika waktu tempuh lava menuju pemain lebih cepat dibandingkan waktu tempuh pemain menuju platform acuan di atasnya.

Dashing berfungsi sebagai mekanisme penyelamatan yang memberikan kesempatan tambahan kepada pemain untuk bertahan hidup pada situasi kritis di mana platform acuan selanjutnya tidak dapat dicapai sebelum lavanya mengenai pemain.

3. Difficulty Dinamis Berdasarkan Kemampuan Pemain

Tingkat kesulitan pada game ini bersifat dinamis dan menyesuaikan dengan kemampuan pemain. Semakin jauh jarak pemain dari lava, semakin tinggi tingkat kesulitan yang diterapkan.

4. Sistem Autentikasi Menggunakan JWT Token

Game ini dilengkapi dengan sistem autentikasi pemain menggunakan JSON Web Token. Sistem ini digunakan untuk memverifikasi akun pemain ketika mengakses fitur yang terhubung dengan backend, seperti penyimpanan skor atau data pemain. Penggunaan JWT memungkinkan proses autentikasi yang aman, ringan, dan stateless sehingga komunikasi client-server dapat berjalan secara efisien.

Interface:

Player dapat mengendalikan Ramid dengan key “A” dan “D” untuk gerakan horizontal, dan “Spacebar” untuk melompat. Terdapat juga icon yang terletak di kiri atas untuk menandakan “Dash” dapat digunakan setelah lava berada cukup dekat dengan Ramid. “Dash” dapat memungkinkan Ramid untuk bergerak dengan cepat ke platform yang awalnya mustahil dicapai karena jarak yang terlalu jauh. “Dash” dapat dilakukan player dengan menggunakan key “Q” untuk ke kiri dan “E” untuk ke kanan.

Art Style:

Dengan style 32-bit, game ini membawakan sifat retro kepada yang memainkannya. Mayoritas asset digunakan dengan akses royalty-free dan tidak melibatkan copyright dari artis manapun.

Music/Sound:

Music: <https://opengameart.org/>

Sound: <https://sfxr.me/>

Video Demo: <https://youtu.be/gQdOk9lIFzk>

Development Roadmap / Launch Criteria:

Platform: Windows (itch.io).

Audience: Everyone

Milestone 1: Mechanics complete - 18/12/25

Milestone 5: GUI complete - 21/12/25

Milestone 2: Backend and Leaderboard System complete - 14/12/25

Milestone 6: UI/UX complete - 21/12/25

Milestone 3: Music and Sound effect complete - 21/12/25

Milestone 7: Polish complete - 21/12/25

Milestone 4: Game Assets complete - 21/12/25

Launch Day: 21/12/25